ECE-9413 Final Report: Negacyclic NTT and SumCheck Prover

Koshiq Hossain (kh3134)

Jack Chou (cc9171)
New York University
ECE-9413

Abstract—This report describes two exact finite-field kernels implemented in JAX: a negacyclic number theoretic transform (NTT) and a SumCheck prover. Both submitted implementations preserve the provided public APIs and pass the required correctness and benchmark checks documented below. The NTT uses a twisted Cooley–Tukey schedule with Montgomery multiplication on the butterfly hot path (jit/vmap, reshape-based stages, optional Pallas kernel off by default). The SumCheck prover uses a general expression evaluator, serialized rounds, uint32 overflow-safe add/sub with widening multiply for products, and MLE updates strictly after each round (no challenge pre-folding). On an NVIDIA A100–40GB (NYU HPC; beyond a typical single-GPU lab card), the NTT reached ≈5.7 Gcoeff/s at N=65536 (batch 1) and remained at multi-Gcoeff/s through N=2^{24}. The required 32-bit SumCheck vars20 GPU medians span ≈0.2–0.6 ms across the four base expressions; the same cases on a representative local CPU are orders of magnitude slower (for example, a*b*c at vars20 is ≈8 ms vs. ≈0.55 ms on the A100 in our tables), so headline end-to-end speedups are dominated by table size and expression degree. Bonus work includes an optional 64-bit core with two-limb Montgomery mod_mul_64 (full core64 tests passing) and three advanced polynomials on vars20. The main failed NTT experiment swapped reshape butterflies for lax.fori_loop: compile time improved on small shapes but runtime roughly doubled once lowering lost contiguous access. The main open limitation is that measurements mix compiler effects (JAX/XLA compile time) with steady-state GPU latency; we separate them where possible, but both remain first-order for SumCheck at vars20.

I. Introduction

NTT and SumCheck are central kernels in proof systems and symbolic cryptography: the NTT accelerates exact polynomial multiplication over finite fields, while SumCheck compresses a Boolean-hypercube exponential sum into a short interactive sequence of low-degree univariate checks. They differ from ordinary dense floating-point GPU work because every operation is modular, overflow is explicit, and reduction rules (Barrett/Montgomery-style arithmetic) often sit inside the innermost loops.

Why these kernels matter here. Both assignments stress the same systems challenge: get a high-level numerical idea (FFT-like recursion; Boolean cube folding) through a modern array compiler without silently changing semantics. JAX makes that easy to prototype but easy to break when integer widening, staging, or loop lowering change the graph.

What we validate and measure. All required public tests pass for both assignments; Tables I–III record the exact uv-based commands. Headline GPU numbers in this report come from batch 1 NTT sweeps and SumCheck –bench runs at vars4/16/20 with –runs 8 –warmup 3 (32-bit required path), executed on an A100–40GB. CPU medians in Table VIII provide a complementary baseline on the same code paths.

Contributions. (1) batched negacyclic NTT with Montgomery butterflies and reshape scheduling; (2) general SumCheck prover for the required expressions without per-expression hard wiring; (3) optional 64-bit track and advanced polynomials as extra credit paths; and (4) benchmark tables and figures traceable to those commands and hardware.

II. Assignment 1: Negacyclic NTT

A. Design and Optimization

The implemented NTT evaluates

$$y[k] = \sum_{n=0}^{N-1} x[n]\psi^{(2k+1)n} \pmod{q}. \tag{1}$$

The code uses the standard negacyclic decomposition: first multiply each input coefficient by the appropriate power of ψ, then run a cyclic Cooley–Tukey NTT. This structure fits the provided tables and keeps the hot path regular across the batch dimension.

The modular arithmetic hot path uses unsigned 32-bit values, with widening only where multiplication requires it. The optimized path converts values into Montgomery form in prepare_tables. The precomputed tables include a pretwisted ψ table and Montgomery-form stage twiddles. The default path uses JAX reshapes and vectorized Gentleman–Sande DIF butterfly stages, which run on both CPU and GPU backends. Reshape isolates contiguous butterfly halves without explicit gather/scatter index arrays, allowing XLA to fuse the elementwise add, subtract, Montgomery multiply, compare, and select operations. An experimental Pallas kernel is kept opt-in, but the final A100 measurements use the stable JAX path. This keeps the public API unchanged while moving table preparation out of the timed benchmark region.

The most useful optimization was moving expensive table conversion and Montgomery constants into prepare_tables. A less attractive approach would have been direct evaluation, which is simpler but has $O(N^2)$ work and does not scale. Another attempted optimization used

TABLE I
Assignment 1 correctness checks (uv –directory assignment1)

Command	Result
uv run pytest	5 passed
uv run pytest –logn 10 –batch 4	5 passed
uv run python -m tests.benchmark –tests –logn 10 –batch 4	5 passed

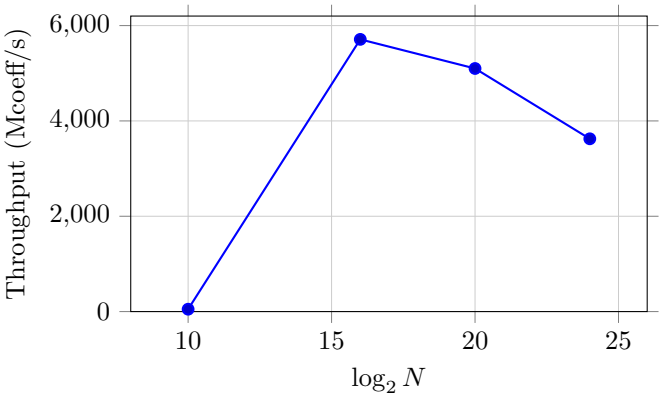

Fig. 1. NTT throughput vs. $\log_2(N)$ (A100–40GB, batch 1).

jax.lax.fori_loop to make the stage loop appear more compact to XLA. It reduced compile time on small RTX experiments, but runtime roughly doubled because dynamic loop shapes forced an index/gather implementation instead of the zero-copy reshape pattern. The final implementation therefore stays with static reshape-based $O(N \log N)$ butterflies.

At the largest measured sizes ($N = 2^{24}$, batch 1), throughput falls from its peak near $N = 2^{16}$, which is consistent with a shift toward memory-bound behavior as arrays outgrow on-chip reuse. At moderate N with batch 1, the kernel is more compute- and launch-latency sensitive before bandwidth fully dominates; we did not build a calibrated roofline plot, but the sweep in Figure 1 makes the qualitative turnover visible.

B. Correctness

Table I matches the course checklist. From the repository root, prefix Assignment 1 commands with uv –directory assignment1; the internal benchmark wrapper changes its working directory accordingly.

C. Performance

Table II lists the measured batch 1 sweep on an A100–40GB. Figure 1 plots the Mcoeff/s column from Table II: it peaks near $N = 65536$, where there is enough parallelism to amortize launches while arrays still fit relatively well for reuse, then softens at $N = 2^{24}$ as traffic dominates.

TABLE II
Assignment 1 A100 scaling sweep, batch 1

$\log_2(N)$	N	Compile (ms)	Median (ms)	Mcoeff/s
10	1024	709	0.081	50
16	65536	1700	0.734	5711
20	1048576	2346	6.58	5100
24	16777216	7031	37.0	3625

III. Assignment 2: SumCheck

A. Design and Optimization

The SumCheck prover supports expressions represented as lists of additive terms, where each term is a list of multiplicative factors. This general evaluator supports the four required base expressions without hard-coding a separate kernel per expression. For an expression of degree d, each round emits $d+1$ evaluations of the round polynomial. Thus the linear expression a emits two values per round, a*b and a*b + c emit three, and a*b*c emits four.

The implementation strictly follows the required round order. For each round, it first computes the round polynomial evaluations using the current tables. Only after that round does it fold the tables using the provided challenge. The code does not precompute challenge-folded tables ahead of time.

a) MLE updates.: Folds use the standard MLE update with full modular multiplication and reduction on the active limb width (no unproven "fast" fold that skips modular reduction). In our implementation the update is scheduled after emitting each round polynomial, matching the course note that challenges are revealed between rounds and that pre-folding for speed would be out of spec.

For the required 32-bit track, multiplication still widens to 64 bits because a 32-bit product can exceed 32 bits. Addition and subtraction, however, now avoid unnecessary 64-bit casts. mod_add_32 detects uint32 wraparound and corrects by adding $2^{32} \bmod q$ before the final conditional subtraction. mod_sub_32 computes the wrapped case as $q-(b-a)$, avoiding a+q overflow. This optimization keeps correctness on the edge cases while reducing the integer width used by common MLE and expression operations.

Ideas carried from Assignment 1 versus SumCheck specifics. The NTT relies on Montgomery multiplication as the workhorse on the butterfly because every stage multiplies by twiddles under a fixed modulus. SumCheck instead spends most of its time scanning and folding large tables and evaluating low-degree round polynomials; the high-value trick on the 32-bit path is to keep add/sub in uint32 with explicit wrap semantics while still widening products. The optional 64-bit track does reuse Montgomery ideas (two-limb mod_mul_64) because native 64-bit products are insufficient to recover full double-width products inside JAX.

The implementation also reduces unused dataflow. The working dictionary is filtered to the polynomial tables that appear in the current expression, so a carries only table a, and a*b carries only a and b. Expression accumulation starts from the first term instead of a zero vector, avoid-

TABLE III
Assignment 2 required 32-bit correctness (uv –directory assignment2)

Command	Result
uv run pytest –bits 32 –num-vars 4	20/20 passed
uv run pytest –bits 32 –num-vars 16	20/20 passed
uv run pytest –bits 32 –num-vars 20	20/20 passed

ing one redundant modular addition for each expression evaluation.

Bonus tracks attempted. (i) Optional 64-bit core: two-limb Montgomery mod_mul_64 and reduction-friendly adds; (ii) advanced polynomials at vars20 (Table VI); (iii) aggregate packaging checks via bash scripts/make_submission.sh. We did not pursue the sumcheck_128 track or the optional hashing-between-rounds track.

The optional 64-bit core required a different approach. Since JAX does not provide a native uint128 type, direct 64-bit multiplication can lose high product bits. The submitted optional path uses two-limb Montgomery multiplication with 32-bit limbs for mod_mul_64, and pairwise overflow-safe modular addition for 64-bit reductions. This made the optional core64 correctness suite pass, although the required report baseline remains the 32-bit track.

B. READMEs

This section records which course artifacts we actually used; skipping a resource is not penalized, but the section is required.

Assignment 1 README. Helpful for the public API surface, the benchmark entry points, and the expectation that table preparation can live outside the hot timed region.

Assignment 2 README and sumcheck_intro.md. The README clarified required expressions and command lines; sumcheck_intro.md was especially useful for round order and the policy that challenges fold tables after a round's polynomial is emitted, not ahead of the protocol.

Other references. No external papers were necessary beyond course material; JAX/XLA documentation was useful for interpreting compile-time vs. runtime effects.

Custom cases. We read the custom-cases notes for ideas but report only the public benchmarks and optional advanced polynomials exercised by our code path.

C. Correctness

Table III lists the three required 32-bit commands. In our environment they were launched from the repository root with uv –directory assignment2. We also ran bash scripts/make_submission.sh as an aggregate bundle before packaging (91 tests passed).

D. Performance

1) Figures: Figures 2 and 3 summarize SumCheck latency trends using representative midpoints of the measured ranges in Tables V and VIII (GPU at vars20; CPU scaling).

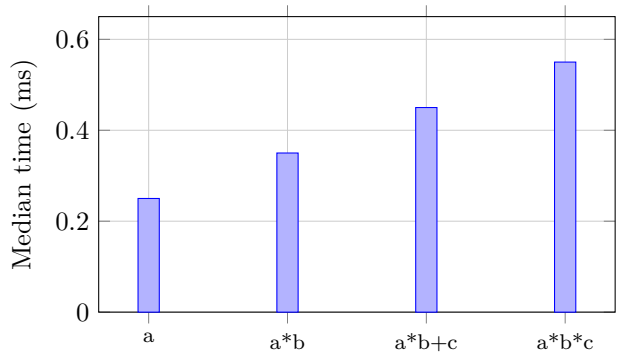

Fig. 2. Representative vars20 GPU medians (A100–40GB, 32-bit) across base expressions; midpoints of Table V ranges.

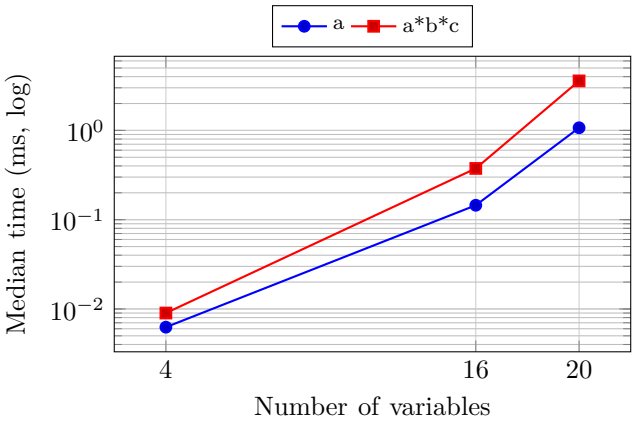

Fig. 3. CPU scaling (representative midpoints from Table VIII) for a vs. a*b*c, illustrating table-driven growth.

2) Benchmarks: Table IV records the required –bench driver lines. Extracted latency/throughput summaries appear in Tables V–VII; compile-time-focused comparisons isolate JAX/XLA staging effects at vars20.

Across vars20 GPU runs, median latency ranks the base expressions as a < a*b < a*b + c < a*b*c. This lines up with polynomial degree (more round-evaluation points per round) and with the extra multiplies in the evaluator, not with a change in table footprint (all cases use the same 2^{20}-row tables for a given variable count). Compile time also grows with evaluator complexity; Table VII shows that implementation choices can move compile time materially even when median run time stays in the same band.

Table VI reports the optional 32-bit advanced polynomials on vars20. These are not required for the base benchmark, but they demonstrate that the general expression evaluator supports repeated factors and multiple additive terms without special-case kernels.

Table VII compares the final implementation against the teammate/reference implementation on the same A100 vars20 benchmark. Both implementations use the same SumCheck protocol and pass correctness. The compile-

TABLE IV
Required SumCheck benchmark commands (uv –directory assignment2)

Command
uv run python -m tests.benchmark –bench –bits 32 –num-vars 4 –runs 8 –warmup 3
uv run python -m tests.benchmark –bench –bits 32 –num-vars 16 –runs 8 –warmup 3
uv run python -m tests.benchmark –bench –bits 32 –num-vars 20 –runs 8 –warmup 3

TABLE V
Assignment 2 vars20 A100 benchmark, 32-bit

Expression	Compile (ms)	Median (ms)	Mpts/s
a	≈2083–2387	0.2–0.3	3330–4290
a*b	≈3792–4175	0.3–0.4	2420–3030
a*b + c	≈4832–5271	0.4–0.5	2080–2530
a*b*c	≈6299–6890	0.5–0.6	1890–2170

time improvement comes from implementation shape: fewer uint32-to-uint64 widening operations in add/sub, less unused table dataflow, and fewer redundant modular additions. Runtime remains close because both versions still scan and fold the same 2^{20}-entry tables.

The local CPU baseline is retained in Table VIII. It shows the same expression ordering, but with much higher vars20 latency.

IV. Cross-Assignment Discussion

The main shared lesson is that modular arithmetic choices dominate both assignments. In the NTT, Montgomery form is effective because each butterfly does many modular multiplications with precomputed twiddles. In SumCheck, the required 32-bit path benefits more from carefully avoiding unnecessary widened addition and subtraction while still widening multiplication. The NTT is more structured: every stage has a regular butterfly layout and a predictable twiddle access pattern. SumCheck is more expression dependent: degree and term count directly change the amount of work per round.

Memory movement also differs. NTT repeatedly permutes and combines arrays, so the schedule and table layout matter. SumCheck repeatedly halves the tables by MLE update; the largest rounds dominate runtime because later rounds quickly become small. This also explains why vars4 benchmark numbers are mostly overhead, while vars20 makes the differences between expressions visible.

Multiplication vs. memory. The NTT is more sensitive to steady-state modular multiplication throughput on butterfly edges (Montgomery mul is the inner loop). SumCheck at vars20 is more sensitive to traversing large tables under a deep JAX graph: multiplication cost still matters—especially as polynomial degree grows—but bandwidth, cache footprint, and XLA compile time surface as first-order effects alongside raw mul latency.

TABLE VI
Assignment 2 vars20 A100 advanced polynomials, 32-bit

Expression	Median (ms)	Mpts/s
a*a*b*b*c	0.7–0.8	1200–1390
a*b*c + d*e	0.5–0.6	1520–1800
a*b*c*g + d*e*g	0.7–0.8	1210–1390

TABLE VII
Assignment 2 vars20 compile-time comparison

Expression	Reference (ms)	Final (ms)	Improvement
a	2457	2173	12%
a*b	4604	3941	14%
a*b + c	5312	5059	5%
a*b*c	7368	6624	10%

V. AI Use Reflection

AI assistance was useful for quickly producing first-pass implementations, debugging edge cases, and designing benchmark commands. The most useful suggestion was to verify the optional 64-bit SumCheck path separately from the required 32-bit path; this exposed a broken 64-bit multiplication routine even though the required submission already passed. A plausible but poor suggestion was to implement 64-bit multiplication with a simple shift-and-add loop. That was correct but too slow for full vars16 and vars20 tests, so it was replaced with a two-limb Montgomery method. The best workflow was to keep AI suggestions tied to public tests and benchmark output rather than trusting code structure alone.

VI. Conclusion

Both assignments pass the course-required public checks cited above. On an A100–40GB, the reshape-based negacyclic NTT reaches ≈5.7 Gcoeff/s at N=65536 (batch 1). The 32-bit SumCheck prover reaches ≈0.2–0.6 ms vars20 medians across the four base expressions on the same GPU, with clear CPU-vs-GPU separation in Figure 3. The most important NTT optimization was keeping butterflies on contiguous reshape paths while relocating Montgomery setup to prepare_tables. The most important SumCheck optimizations were protocol-faithful scheduling (no challenge pre-folding) and reducing unnecessary widening in the 32-bit add/sub path. Main limitation: first-run JAX/XLA compilation remains a large fraction of wall time at vars20, and our headline GPU numbers come from HPC-class hardware rather than a minimal lab GPU.

TABLE VIII
Assignment 2 scaling by variable count, representative CPU medians

Expression	vars4 (ms)	vars16 (ms)	vars20 (ms)
a	0.005–0.006	0.13–0.16	1.05–1.09
a*b	0.007–0.011	0.36–0.39	3.47–3.70
a*b + c	0.007–0.008	0.45–0.54	6.01–6.41
a*b*c	0.007–0.008	0.54–0.59	7.98–8.45